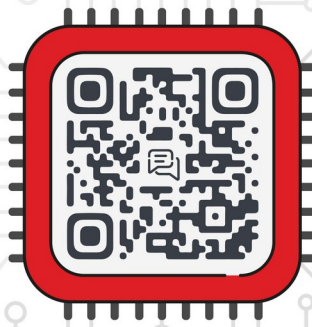




نظري

5



8



960 sp

كلية الهندسة المعلوماتية

السنة الرابعة - النظم والشبكات

شرح بروتوكول HTTP وتصميم البروتوكولات

د. غسان شذور



محتوى مجاني غير مخصص للبيع التجاري
RB Informatics

1/12/2025

برمجة تطبيقات شبكية

السلام عليكم ورحمة الله وبركاته

يمثل تصميم البروتوكولات حجر الزاوية في بناء أي نظام شبكي، إذ تحدد هذه البروتوكولات المعايير والقواعد التي يمكن الأجهزة والتطبيقات المختلفة من التواصل وفهم بعضها البعض. بدون بروتوكولات محددة جيداً، تتحول الشبكات إلى فوضى من البيانات غير المترابطة.

بروتوكول HTTP

1. نظرة عامة على طبقة التطبيق (Application Layer)

تُعد طبقة التطبيق الطبقة الأقرب للمستخدم في نموذج الشبكات (نموذج TCP/IP)، حيث تتفاعل التطبيقات مباشرة مع البروتوكولات لتوفير الخدمات. هي الواجهة التي يتم من خلالها تبادل البيانات بين البرامج المختلفة عبر الشبكة. المجالات الرئيسية التي تغطيها هذه الطبقة:

1. مبادئ تطبيقات الشبكة: المفاهيم الأساسية التي تحكم عمل التطبيقات عبر الشبكة.
2. الويب و HTTP: البروتوكول الرئيسي الذي يشغل الويب اليوم.
3. البريد الإلكتروني: بروتوكولات إرسال واستقبال الرسائل (SMTP, IMAP).
4. نظام أسماء النطاقات (DNS): الخدمة التي تترجم أسماء النطاقات إلى عناوين IP.
5. تطبيقات الند للند (P2P): الاتصال المباشر بين الأجهزة دون خادم مركزي.
6. بث الفيديو وشبكات توزيع المحتوى (CDN): تقنيات مشاهدة الفيديو بكفاءة وتوزيع المحتوى جغرافياً.

2. مكونات صفحة الويب

تتكون صفحة الويب من عدة كائنات (Objects)، مثل ملفات HTML، صور JPEG، وتطبيقات Java. ملف HTML الأساسي: هو الملف الذي يتضمن إشارات (References) إلى الكائنات الأخرى المضمنة في الصفحة. عنوان URL: هو المعرّف الفريد لكل كائن، ويتكون من اسم المضيف (Host Name) واسم المسار (Path Name).

3. ما هو بروتوكول HTTP؟

HTTP هو بروتوكول طبقة التطبيق للويب، ويعتمد على نموذج العميل/الخادم (Client/Server).

الدور	الطرف
المتصفح الذي يطلب ويستقبل ويعرض كائنات الويب.	العميل (Client)
خادم الويب الذي يرسل الكائنات استجابة للطلبات.	الخادم (Server)

الخصائص التقنية الأساسية:

يعتمد على TCP: يستخدم HTTP بروتوكول TCP لضمان اتصال موثوق وموجه. يتم الاتصال عادة عبر المنفذ 80 (HTTP) أو 443 (HTTPS).
عديم الحالة (Stateless): لا يحتفظ الخادم بأي معلومات عن الطلبات السابقة للعميل. هذا يبسط تصميم الخادم ويجعله أكثر قابلية للتوسع.

4. أنواع اتصالات HTTP

التأثير على الأداء	الوصف	النوع
يزيد من الحمل (Overhead) ويؤخر تحميل الصفحة بسبب كثرة فتح وإغلاق الاتصالات.	يتم فتح اتصال TCP جديد لكل كائن يتم طلبه، ويُغلق بعد إرساله.	HTTP غير المستمر (Non-persistent)
يوفر أداءً أفضل بكثير عن طريق تقليل عدد مرات فتح الاتصالات.	يتم إرسال كائنات متعددة عبر اتصال TCP واحد بين العميل والخادم. هو الوضع الافتراضي في HTTP/1.1.	HTTP المستمر (Persistent)

5. بنية رسالة الطلب (Request Message)

رسالة الطلب هي رسالة نصية بتنسيق ASCII وتتكون من أربعة أجزاء رئيسية:

1. سطر الطلب (Request Line): يتضمن الطريقة (Method)، المسار (URI)، وإصدار البروتوكول. مثال: GET /index.html HTTP/1.1
2. أسطر الرأس (Header Lines): معلومات إضافية مثل Host و User-Agent. مثال: Host: www.example.com
3. سطر فارغ: يشير إلى نهاية أسطر الرأس.
4. جسم الكيان (Entity Body): اختياري، ويستخدم لإرسال البيانات إلى الخادم (مثل بيانات النماذج في طلب POST).

طرق الطلب (Methods) الأكثر شيوعاً:

الطريقة	الغرض
GET	طلب كائن من الخادم.
POST	إرسال بيانات من العميل إلى الخادم (مثل ملء نموذج).
HEAD	طلب معلومات عن الكائن (الرؤوس فقط) دون تحميل الكائن نفسه.
PUT	تحميل ملف جديد إلى الخادم.
DELETE	حذف كائن من الخادم.

6. بنية رسالة الاستجابة (Response Message)

تتكون رسالة الاستجابة من ثلاثة أجزاء رئيسية:

1. سطر الحالة (Status Line): يتضمن الإصدار، رمز الحالة، وعبارة الحالة. مثال: HTTP/1.1 200 OK
2. أسطر الرأس (Header Lines): معلومات عن الخادم والاستجابة (مثل التاريخ، نوع المحتوى، وطوله).
3. البيانات (Data): الكائن المطلوب (مثل ملف HTML أو صورة).

أبرز رموز الحالة (Status Codes):

الفئة	النطاق	المعنى	أمثلة شائعة
1xx	100-199	معلوماتي	Continue 100
2xx	200-299	نجاح (Success)	OK 200 (نجاح الطلب والكائن موجود)
3xx	300-399	إعادة توجيه	301 Moved Permanently (نقل الكائن بشكل دائم)
4xx	400-499	خطأ العميل (Client Error)	400 Bad Request (خطأ في صياغة الطلب)، 404 Not Found (الكائن غير موجود)
5xx	500-599	خطأ الخادم (Server Error)	503 HTTP Version Not Supported

7. كيف يحافظ HTTP على الحالة؟

نظراً لأن HTTP بروتوكول عديم الحالة، يتم استخدام الكوكيز (Cookies) لتتبع تفاعلات المستخدم، مثل سلة التسوق أو تسجيل الدخول. المكونات الأربعة لآلية الكوكي:

1. رأس Set-Cookie: يرسله الخادم في رسالة الاستجابة لتعريف المستخدم لأول مرة.
2. رأس Cookie: يرسله العميل في الطلبات اللاحقة لنفس الموقع.
3. ملف الكوكي: يحفظه المتصفح على جهاز المستخدم.
4. قاعدة بيانات خلفية: في موقع الويب لربط مُعرّف الكوكي بمعلومات حالة المستخدم.

8. لماذا احتجنا إلى HTTP/2؟

كان الهدف الرئيسي من HTTP/2 هو تقليل التأخير في الطلبات المتعددة. واجه HTTP/1.1 قيوداً هيكلية أدت إلى مشاكل في الأداء:

1. حجب مقدمة السطر (HOL Blocking): في HTTP/1.1، يجب أن ينتظر الكائن الصغير خلف الكائنات الكبيرة ليتم إرساله، حتى لو كان جاهزاً. يحدث هذا لأن الطلبات والاستجابات تُعالج بالترتيب الذي تصل به (FCFS) عبر اتصال TCP واحد.
2. الحاجة إلى اتصالات TCP متعددة: لتجاوز مشكلة HOL Blocking، كان المتصفح يفتح اتصالات TCP متعددة، مما يزيد من استهلاك الموارد والحمل على الشبكة.

9. مميزات HTTP/2

جاء HTTP/2 (RFC 7540) لتحسين الأداء مع الحفاظ على التوافق مع الإصدار السابق:

1. الإرسال المتعدد (Multiplexing) عبر إطار (Frames): هذه هي الآلية الأساسية لمعالجة HOL Blocking. يتم تقسيم الكائنات إلى إطارات (Frames) صغيرة، مما يسمح بإرسال أجزاء من طلبات واستجابات متعددة بالتوازي عبر اتصال TCP واحد. هذا يلغي الحاجة إلى اتصالات متعددة ويحسن كفاءة استخدام الشبكة بشكل كبير.
2. أولوية الكائنات (Stream Prioritization): يمكن للعميل تحديد أولوية إرسال الكائنات المطلوبة، مما يضمن تحميل العناصر الأكثر أهمية (مثل CSS) قبل العناصر الأقل أهمية (مثل الصور).
3. دفع الخادم (Server Push): يمكن للخادم إرسال موارد يتوقع أن يحتاجها العميل (مثل ملفات CSS أو JavaScript) قبل أن يطلبها العميل. هذا يقلل من زمن الرحلة الذهاب والإياب (RTT) ويحسن سرعة التحميل.
4. ضغط الرؤوس (Header Compression): يستخدم HTTP/2 خوارزمية HPACK لضغط رؤوس HTTP، والتي غالباً ما تكون متكررة، مما يقلل من حجم البيانات المنقولة ويسرع عملية الاتصال.

ملاحظة : يمثل الانتقال إلى HTTP/2 قفزة نوعية في كفاءة الويب، حيث تم حل مشاكل التأخير الهيكلية مع الحفاظ على بساطة البروتوكول الأساسي. هذه المفاهيم ضرورية لأي مطور أو مهتم بالبنية التحتية للويب.

تصميم البروتوكولات (الأسس النظرية)

يمثل تصميم البروتوكولات حجر الزاوية في بناء أي نظام شبكي، إذ تحدد هذه البروتوكولات المعايير والقواعد التي تمكن الأجهزة والتطبيقات المختلفة من التواصل وفهم بعضها البعض. بدون بروتوكولات محددة جيداً، تتحول الشبكات إلى فوضى من البيانات غير المترابطة.

1. لماذا نحتاج إلى بروتوكولات على مستوى التطبيق؟

عندما نتحدث عن تصميم بروتوكولات لتطبيقات الشبكة، فإننا نتقل من المستويات الدنيا (مثل TCP/IP) إلى المستويات العليا. تعتمد بروتوكولاتنا على الخدمات التي توفرها طبقات النقل (مثل TCP و UDP)، ولكنها تضيف طبقة من التنظيم والمعنى فوقها.

قد يتساءل البعض عن الحاجة إلى بروتوكولات خاصة بالتطبيقات طالما لدينا TCP/IP. تكمن الإجابة في أن TCP، على الرغم من موثوقيته، يتعامل مع البيانات ك تدفق من البايتات (Byte Stream) وليس ك رسائل (Messages) ذات حدود واضحة. هذا يطرح عدة مشكلات حرجة:

1. حدود الرسائل (Message Boundaries): كيف يمكننا معرفة أين تنتهي رسالة وتبدأ أخرى في تدفق البتات المستمر؟
2. ارتباط الطلب/الاستجابة (Request/Response Correlation): في أنظمة العميل/الخادم، كيف نربط استجابة معينة بطلب محدد، خاصة إذا كانت هناك طلبات متعددة معلقة في نفس الوقت؟
3. دلالات الأخطاء (Error Semantics): كيف نبليغ عن الأخطاء بطريقة مفهومة للتطبيق، وليس مجرد أخطاء شبكة؟
4. إدارة الحالة (State Management): كيف نحافظ على حالة معينة عبر تفاعلات متعددة بين العميل والخادم؟

ملاحظة: تحل بروتوكولات التطبيقات (مثل HTTP، SMTP، WebSocket) هذه المشكلات من خلال توفير بنية محددة للرسائل ودلالات واضحة للتفاعلات.

2. تمثيل البيانات والتسلسل (Serialization)

التسلسل هو عملية تحويل كائن برمجي معقد (يمثل رسالة أو مستخدماً، على سبيل المثال) من شكله في الذاكرة إلى تسلسل من البتات يمكن تخزينه أو نقله عبر الشبكة. العملية العكسية تسمى إلغاء التسلسل (Deserialization)، حيث يتم إعادة بناء الكائن الأصلي من تسلسل البتات في الطرف المستقبل.

لماذا هو ضروري؟ الشبكات تنقل البتات فقط، ولا تفهم مفهوم الكائنات البرمجية. لكي تتواصل الأنظمة المكتوبة بلغات برمجة مختلفة أو تعمل على منصات متباينة، يجب أن تتفق على تنسيق مشترك لتمثيل البيانات. كما أن التسلسل يُمكن من الثبات (Persistence)، أي حفظ حالة الكائنات.



3. مقارنة أساليب التسلسل

الميزة	التنسيقات النصية (Text-Based)	التنسيقات الثنائية (Binary Formats)
أمثلة	JSON, XML	Protocol Buffers (Protobuf), Thrift
قابلية القراءة	سهولة القراءة والفهم (ميزة قوية لتصحيح الأخطاء البشرية)	غير قابلة للقراءة المباشرة
حجم البيانات	حجم أكبر مقارنة بالتنسيقات الثنائية (بسبب تكرار أسماء الحقول)	مدمجة (Compact)، تقلل من حجم البيانات المنقولة
سرعة التحليل (Parsing)	أبطأ نسبياً	أسرع بكثير
البيني (Interoperability)	ممتازة، مدعومة عالمياً في جميع اللغات	جيدة جداً، تتطلب مولدات شفرة (Code Generators)

نقد التسلسل الخاص بلغة Java:

على الرغم من سهولة استخدامه، فإن التسلسل المدمج في لغة Java يواجه قيوداً تجعله غير مناسب لبروتوكولات الشبكة عالية الأداء:

الخصوصية اللغوية (Java-specific): يعمل فقط مع كائنات Java، مما يحد من قابليته للتشغيل البيني مع تطبيقات مكتوبة بلغات أخرى (مثل Python أو ++C).

مشاكل الإصدار (Versioning Issues): إذا تغير هيكل الفئة (كإضافة أو حذف حقل) بعد تسلسل الكائن، قد تحدث مشاكل عند محاولة إلغاء تسلسله بإصدار مختلف من الفئة.

عبء الأداء (Performance Overhead): يمكن أن يكون بطيئاً ويستهلك الكثير من الموارد، خاصة مع الكائنات الكبيرة أو المعقدة.

ملاحظة: لهذه الأسباب، غالباً ما نلجأ إلى أساليب تسلسل محايدة للغة مثل JSON أو Protobuf عند تصميم بروتوكولات الشبكة التي تتطلب أداءاً عالياً ومرونة.



4. مشكلة تأطير الرسائل

تعد مشكلة تأطير الرسائل تحدياً محورياً عند استخدام بروتوكولات النقل القائمة على تدفق البتات مثل TCP. كما ذكرنا، لا يوفر TCP حدوداً متأصلة للرسائل. إذا أرسل العميل رسالتين متتاليتين ("HELLO" ثم "WORLD")، قد يستقبل الخادم هذه البيانات ككتلة واحدة ("HELLOWORLD") أو مقسمة بشكل عشوائي ("HEL" + "LOWORLD").

الحل يكمن في التأطير على مستوى التطبيق (Application-level Framing)، حيث يجب على البروتوكول الذي نصممه أن يضيف معلومات إضافية إلى تدفق البتات لتمكين الطرف المستقبل من تحديد حدود الرسائل بشكل صحيح.

5. تقنيات تأطير الرسائل

أ. التأطير المسبوق بالطول (Length-Prefixed Framing):

الفكرة: قبل إرسال حمولة الرسالة (Payload)، يتم إرسال حقل ثابت الحجم يحدد طول حمولة الرسالة بالبتات. الهيكل: [طول الحمولة] [الحمولة].

المزايا: كفاءة عالية، تخصيص فعال للذاكرة، ولا حاجة للهروب (Escaping).
العيوب: يتطلب تحديداً مسبقاً لحجم حقل الطول (عادة 4 بايت)، مما يحدد الحد الأقصى لحجم الرسالة.

ب. التأطير القائم على المحدد (Delimiter-Based Framing):

الفكرة: يتم استخدام تسلسل بتات خاص (Delimiter - محدد) للإشارة إلى نهاية الرسالة. الهيكل: [الحمولة] [المحدد].

المزايا: بسيط وسهل التنفيذ، ومناسب للبروتوكولات النصية (مثل HTTP/1.1 الذي يستخدم \r\n\r\n لإنهاء الرأس).

العيوب: يجب تطبيق آلية الهروب (Escaping) إذا ظهر المحدد داخل حمولة الرسالة نفسها.

ج. الرؤوس ذات الحجم الثابت (Fixed-Size Headers):

الفكرة: يتم استخدام رأس رسالة بحجم ثابت يحتوي على معلومات أساسية مثل نوع الرسالة وطول الحمولة. الهيكل: [رأس الرسالة الثابت] [الحمولة].

المزايا: يجمع بين تحديد النوع والطول في خطوة واحدة، مما يسهل على الطرف المستقبل تحديد كيفية قراءة باقي الرسالة.

6. نمط الطلب/الاستجابة (Request/Response Pattern)

يعد نمط الطلب/الاستجابة هو الأكثر شيوعاً في تصميم بروتوكولات الشبكة، وهو الأساس الذي بنيت عليه معظم تفاعلات العميل والخادم.

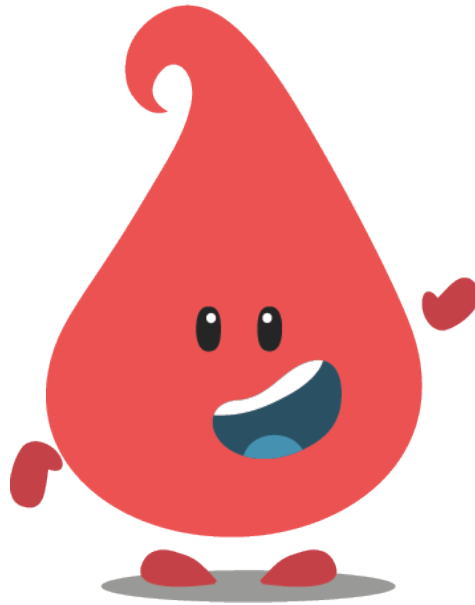
آلية العمل:

1. يرسل العميل طلباً (Request) إلى الخادم.
2. يعالج الخادم الطلب.
3. يرسل الخادم استجابة (Response) إلى العميل.



أمثلة تطبيقية: HTTP (يرسل المتصفح طلب GET ويستجيب الخادم برمز حالة ومحتوى الصفحة)، قواعد البيانات (يرسل العميل استعلام QUERY وتستجيب قاعدة البيانات بمجموعة النتائج)، RPC (يستدعي العميل دالة مثل login ويستجيب الخادم بنتيجة العملية).

خصائص النمط: التزامن (Synchronous) حيث ينتظر العميل الاستجابة غالباً، وارتباط الرسائل (Correlated Messages) حيث يجب ربط الاستجابة بالطلب الأصلي باستخدام مُعرِّف فريد.



THE END